

Compute Services

Prof. Arnab Mandal

Assistant Professor

CSE Department

Content

1. IaaS.....	3
2. Virtual Machines in the Cloud.....	4
3. Elastic Load Balance	5
4. Launching an Amazon EC2 instance.....	6
5. Routing Table.....	12
6. NAT.....	16
8. VPC.....	19
9. Load balancer	25
12. Security Groups	28
13. VPC security.....	29
14. Network ACLS.....	30
15. Route 53.....	31
16. Amazon CloudFront.....	32
17. Launch Web Server	33

Infrastructure as a Service (IaaS):

In the IaaS model, the cloud provider manages the physical hardware, while the consumer manages the operating system and everything above it.

Virtual Machines in the Cloud (Amazon EC2)

Definition: Amazon Elastic Compute Cloud (EC2) provides secure, resizable compute capacity. It reduces the time required to obtain and boot new server instances to minutes.

The Virtual Server Mechanism : A simulated physical server created via virtualization software. In AWS, this is the **EC2 Instance**.

Key Components:

- **AMI (Amazon Machine Image):** A template containing the software configuration (OS, application server, and applications).
- **Instance Types:** Optimized for different use cases (e.g., **Compute Optimized** for high-performance processors, **Memory Optimized** for large data sets in memory).
- **EBS (Elastic Block Store):** High-performance block storage designed for use with EC2 for both throughput and transaction-intensive workloads.

Elastic Load Balancing (ELB)

The Load Balancer Mechanism : A runtime artifact that distributes workloads across multiple IT resources to prevent over-utilization and improve availability.

AWS Implementation:

Application Load Balancer (ALB): Best for HTTP and HTTPS traffic; operates at Layer 7 (Application Layer).

Network Load Balancer (NLB): Best for TCP, UDP, and TLS traffic where extreme performance is required; operates at Layer 4 (Transport Layer).

Health Checks: ELB only routes traffic to "Healthy" instances, ensuring high availability.

Launching an Amazon EC2 Instance (Step-by-Step)

1. **Select AMI:** Choose the OS (e.g., Amazon Linux 2, Windows Server 2022).
2. **Select Instance Type:** Choose hardware specs (e.g., t3.micro).
3. **Configure Instance Details:** Assign to a **VPC** (Virtual Private Cloud) and **Subnet**.
4. **Add Storage:** Define the size and type of the EBS volume.
5. **Add Tags:** Metadata for organization (e.g., Key: Name, Value: WebServer).
6. **Configure Security Group:** A virtual firewall to control inbound and outbound traffic.
7. **Review and Launch:** Select/Create a **Key Pair** (Private key file) to enable secure SSH/RDP access.

Serverless Computing

Serverless is an execution model where the cloud provider is responsible for executing a piece of code by dynamically allocating the resources.

AWS Lambda

Definition: A "Function as a Service" (FaaS) that runs code in response to events and automatically manages the underlying compute resources.

The Multi-Device Broker : Used to facilitate communication between diverse cloud-based IT resources and the Lambda function.

Characteristics of AWS Lambda

Event-Driven: Triggered by other AWS services (S3, DynamoDB, Kinesis, API Gateway).

Sub-second Metering: You are charged for every 1ms your code executes and the number of times your code is triggered.

Stateless: No data is stored within the function between executions. State must be stored in external services like Amazon S3 or DynamoDB.

Automatic Scaling: Scales precisely with the size of the workload, from a few requests per day to thousands per second.

Creating and Configuring a Lambda Function

1. **Define Function Name and Runtime:** (e.g., Python 3.12, Node.js 20).
2. **Assign Execution Role:** An IAM (Identity and Access Management) role that grants the function permission to access other AWS resources.
3. **Add Trigger:** Select the source (e.g., an S3 bucket "PutItem" event).
4. **Configure Memory:** You allocate memory (128 MB to 10 GB); AWS allocates CPU power linearly proportional to the memory.
5. **Set Timeout:** Maximum time a function can run is 15 minutes (900 seconds).

Auto Scaling

The Automated Scaling Listener : A service logic that monitors the workload of cloud resources and automatically triggers scaling based on pre-defined parameters.

AWS Implementation (EC2 Auto Scaling):

Dynamic Scaling: Responds to changing demand (e.g., scaling out when CPU usage hits 70%).

Predictive Scaling: Uses machine learning to schedule the right number of EC2 instances based on predicted traffic.

Fleet Management: Automatically replaces impaired instances to maintain the desired count.

Serverless Computing: AWS Lambda

Serverless allows you to build and run applications without thinking about servers. AWS handles the provisioning, scaling, and management of the infrastructure.

Characteristics of AWS Lambda

- **Event-Driven:** Runs code only when triggered (e.g., an image upload to S3, a new database entry, or an HTTP request).
- **Pay-per-use:** You are charged based on the number of requests and the duration your code runs.
- **Scalability:** Lambda scales automatically by running code in response to each trigger.
- **Stateless:** Each execution happens in a fresh environment; data must be stored in external services like S3 or DynamoDB.

Platform as a Service (PaaS)

PaaS provides a framework that allows developers to build and deploy applications without having to manage the underlying infrastructure.

1. AWS Elastic Beanstalk

Definition: An orchestration service used for deploying and scaling web applications developed with Java, .NET, PHP, Node.js, Python, Ruby, Go, and Docker.

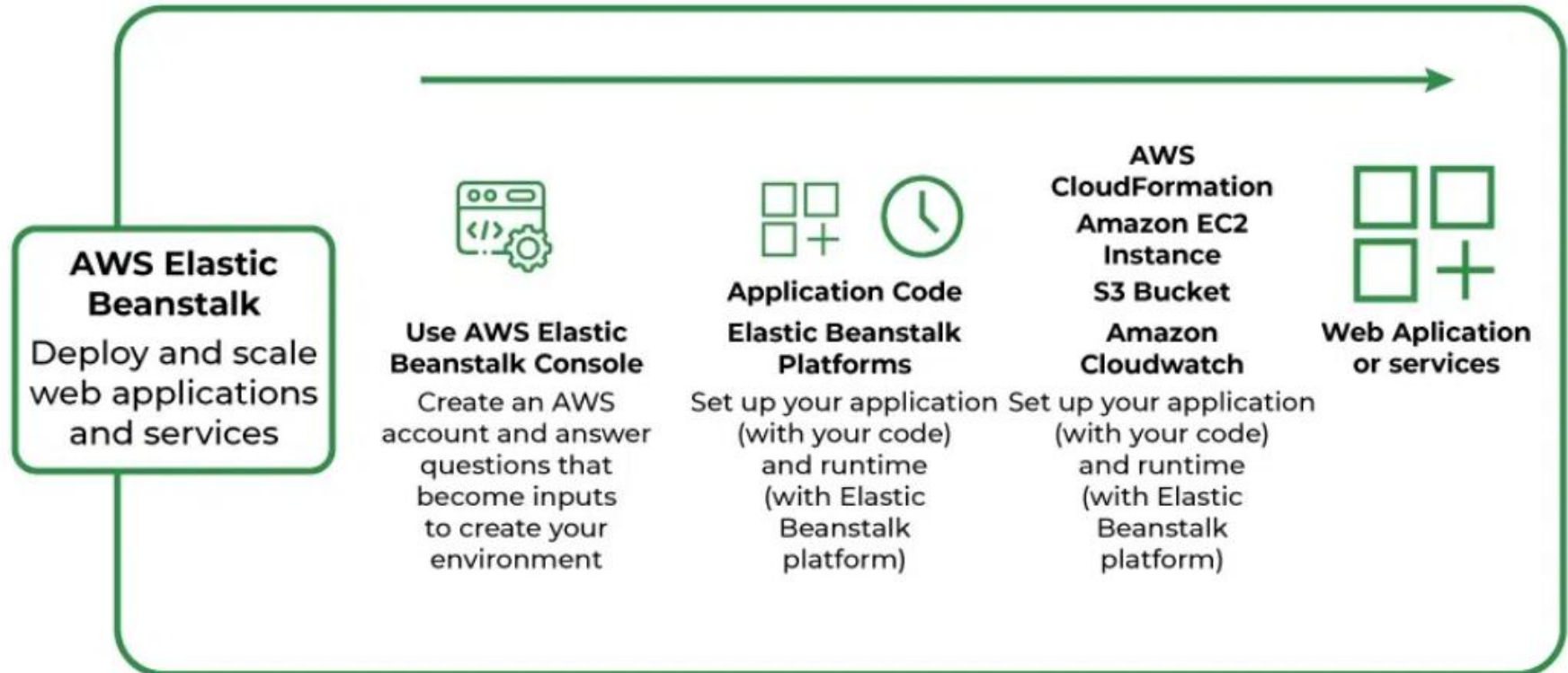
The Resource Management Console : Elastic Beanstalk acts as the unified interface to coordinate various mechanisms (EC2, ELB, ASG, CloudWatch) as a single application environment.

Deploying web applications can be complex, involving servers, load balancers, and scaling. AWS Elastic Beanstalk simplifies this by automating provisioning, scaling, and monitoring. Acting as a Platform as a Service (PaaS), it lets developers focus on writing code rather than managing servers and configurations.

- It automates deployment of resources like EC2 instances, load balancers, and databases.
- Handles scaling automatically to manage varying traffic loads efficiently.
- Provides health monitoring and application versioning for reliability and quick updates.
- Abstracts infrastructure management, letting developers focus on application development

Elastic Beanstalk Working

Elastic Beanstalk is a fully managed service provided by AWS that makes it easy to deploy and manage applications in the cloud without worrying about the underlying infrastructure. First, create an application and select an environment, configure the environment, and deploy the application.



Application: This is a container for your entire project, including all its different environments and application versions.

Application Version: This refers to a specific, labeled iteration of your deployable code, such as a .ZIP or .WAR file stored in an S3 bucket.

Environment: This is the most important concept. An **environment** is the collection of AWS resources that Elastic Beanstalk provisions and configures to actually run your application version.

When you launch an environment, Elastic Beanstalk provisions the following resources for you:

Amazon EC2 Instances: The virtual servers that run your application code.

Auto Scaling Group: Automatically adjusts the number of EC2 instances up or down based on the load.

Elastic Load Balancer: Distributes incoming traffic across your fleet of EC2 instances to ensure high availability.

Security Group: A virtual firewall to control inbound and outbound traffic to your instances.

Amazon S3 Bucket: Stores your application code, logs, and other assets.

Amazon CloudWatch Alarms: Monitors key metrics for your environment.

Key Features and Benefits

Managed Infrastructure: Elastic Beanstalk automatically provisions and manages the underlying AWS resources, freeing you from infrastructure complexity.

Auto Scaling: The environment can automatically scale to handle peaks in workload or traffic while minimizing costs during quiet periods.

Health Monitoring: Provides a simple and clear health dashboard (Green, Yellow, Red) based on a variety of factors, giving you an at-a-glance view of your application's status.

Deployment Strategies: Supports various deployment strategies like "All at once," "Rolling," "Rolling with additional batch," and "Immutable," allowing you to update your application safely with minimal to zero downtime.

Developer Productivity: This is the main benefit. Developers can focus on writing code and use familiar tools like Git or an IDE to push deployments, dramatically speeding up the development-to-production lifecycle.

No Additional Cost: There is no extra charge for using Elastic Beanstalk. You only pay for the underlying AWS resources (like the EC2 instances and Load Balancer) that it provisions.

How to Deploy an Application Using Elastic Beanstalk

Step 1: Visit the AWS Console and type "Elastic Beanstalk" into the search bar. After selecting Elastic Beanstalk from the drop-down menu below, you will be taken to the Elastic Beanstalk page where you can click "**Create application**" to launch a sample application using Elastic Beanstalk.

The screenshot displays the AWS Management Console interface. At the top, the search bar contains 'Elastic Beanstalk'. The left sidebar shows the navigation menu with categories like EC2 Dashboard, Instances, Images, and Elastic Block Store. The main content area shows search results for 'Elastic Beanstalk'. The 'Services' section is highlighted with a green box, showing 'Elastic Beanstalk' with the description 'Run and Manage Web Apps'. Below this, there are sections for 'Top features' and 'Features'. The 'Features' section lists 'Applications' and 'Elastic Beanstalk feature'. On the right side, there is a panel with 'Actions' and 'Launch instances' buttons, and a table showing alarm status and availability zones.

Search results for 'Elastic Beanstalk'

Services [See all 12 results](#)

- Elastic Beanstalk** ☆
Run and Manage Web Apps

Top features
[Applications](#) [Environments](#)

- Elastic Transcoder** ☆
Easy-to-Use Scalable Media Transcoding
- Elastic Container Service** ☆
Highly secure, reliable, and scalable way to run containers
- Elastic Container Registry** ☆
Fully-managed Docker container registry : Share and deploy container software, publ...

Features [See all 26 results](#)

- Applications**
 - Elastic Beanstalk feature

Actions [Launch instances](#)

Alarm status	Availability Zone	Public IP
No alarms +	ap-south-1b	-
No alarms +	ap-south-1b	-

The screenshot displays the AWS Elastic Beanstalk console. The top navigation bar includes the AWS logo, a 'Services' menu, a search bar, and user information (Mumbai, naveen). The left sidebar shows the 'Elastic Beanstalk' section with sub-links for 'Environments', 'Applications', and 'Change history'. The main content area features the heading 'Amazon Elastic Beanstalk' and the subtitle 'End-to-end web application management.' Below this, a paragraph describes the service as an easy-to-use tool for deploying and scaling web applications on various servers. A 'Get started' section contains the text 'Easily deploy your web application in minutes.' and a prominent orange 'Create Application' button, which is highlighted with a green rectangular border. Further down, a 'How it works' section explains the automated deployment and scaling process, followed by a 'Pricing' section stating that there is no additional charge for Elastic Beanstalk itself, as users pay for the underlying AWS resources. The footer contains links for 'CloudShell', 'Feedback', and 'Language', along with copyright information and links to 'Privacy', 'Terms', and 'Cookie preferences'.

aws Services Search [Alt+S] Mumbai naveen

Elastic Beanstalk

Environments
Applications
Change history

Amazon Elastic Beanstalk

End-to-end web application management.

Amazon Elastic Beanstalk is an easy-to-use service for deploying and scaling web applications and services developed with Java, .NET, PHP, Node.js, Python, Ruby, Go, and Docker on familiar servers such as Apache, Nginx, Passenger, and IIS.

Get started

Easily deploy your web application in minutes.

Create Application

How it works

You simply upload your code and Elastic Beanstalk automatically handles the deployment, from capacity provisioning, load balancing, and automatic scaling to web application health monitoring, with ongoing fully managed patch and security updates. [Learn more](#)

Pricing

There's no additional charge for Elastic Beanstalk. You pay for Amazon Web Services resources that we create to store and run your web application, like Amazon S3 buckets and Amazon EC2 instances.

CloudShell Feedback Language © 2023, Amazon Web Services India Private Limited or its affiliates. Privacy Terms Cookie preferences

Step 3: In this step, we are going to configure the necessary Application information like Name, and key-value pairs and select the platform in which the language the application is developed has to be deployed, the **platform branch**, **platform version**, and **Application code**.

Example: In this example, we are going to deploy the sample Java application which is provided by AWS itself select all the options as shown in the below screenshot.

If you want to deploy your own application code then select the option "**Upload your code**" It needs to be in form of a ZIP or WAR file which is compulsory and needs to be in your local computer or S3 bucket and a version label which your going to provide must be unique.

 Services [Alt+S]

Mumbai naveen

Elastic Beanstalk

Environments

Applications

Change history

Elastic Beanstalk > Getting started

Create a web app

Create a new application and environment with a sample application or your own code. By creating an environment, you allow Amazon Elastic Beanstalk to manage Amazon Web Services resources and permissions on your behalf. [Learn more](#)

Application information

Application name

Gfg-application

Up to 100 Unicode characters, not including forward slash (/).

Application tags

Apply up to 50 tags. You can use tags to group and filter your resources. A tag is a key-value pair. The key must be unique within the resource and is case-sensitive. [Learn more](#)

Key	Value	
		Remove tag

Add tag

50 remaining

CloudShell Feedback Language

© 2023, Amazon Web Services India Private Limited or its affiliates. Privacy Terms Cookie preferences

Elastic Beanstalk ✕

Environments
Applications
Change history

Platform

Platform
Java ▼

Platform branch
Corretto 17 running on 64bit Amazon Linux 2 ▼

Platform version
3.4.6 (Recommended) ▼

Application code

☒ **Sample application**
Get started right away with sample code.

☐ **Upload your code**
Upload a source bundle from your computer or copy one from Amazon S3.

Cancel Configure more options **Create application**

CloudShell Feedback Language © 2023, Amazon Web Services India Private Limited or its affiliates. Privacy Terms Cookie preferences

Step 4: In this step, We do not need to do anything AWS Elastic Beanstalk will take care of the resource required to deploy the web application like

EC2 instance

Security Group

S3 bucket

Environment

Load Balancer Listener

CloudWatch Alarms

Auto Scaling Group

Elastic load balancer (ELB)

```
8:40pm Successfully launched environment: GettingStarted-env
8:39pm Environment health has transitioned from Pending to Ok. Initialization completed 16 seconds ago and took 5 minutes.
8:36pm Added instance [i-045eb69a24818d1d4] to your environment.
8:36pm Waiting for EC2 instances to launch. This may take a few minutes.
8:35pm Created EIP: 34.230.236.246
8:34pm Created security group named:
      eb-dv-e-sbj4gzf2dm-stack-AWSEBSecurityGroup-KATGTRO6V1J9
8:34pm Environment health has transitioned to Pending. Initialization in progress (running for 8 seconds). There are no instances.
8:34pm Using elasticbeanstalk-us-east-1-270205402845 as Amazon S3 storage bucket for environment data.
8:34pm createEnvironment is starting.
```

Step 5: In the screenshot below, under "**Gfgapplication-env**," we can see the URL of the application, which can be accessed through the internet. We can also see that the health is marked in "**green color, OK**," indicating that our application has been successfully deployed and has not encountered any difficulties. The resources or full architecture developed by AWS Beanstalk to deploy our application are visible in the **Recent Events** section.

Elastic Beanstalk

Environments > Gfgapplication-env

Gfgapplication-env
Gfgapplication-env.eba-baggs2nm.ap-south-1.elasticbeanstalk.com [e-y6mnyfaqdz]
Application name: Gfg-application

Health
Ok
Causes

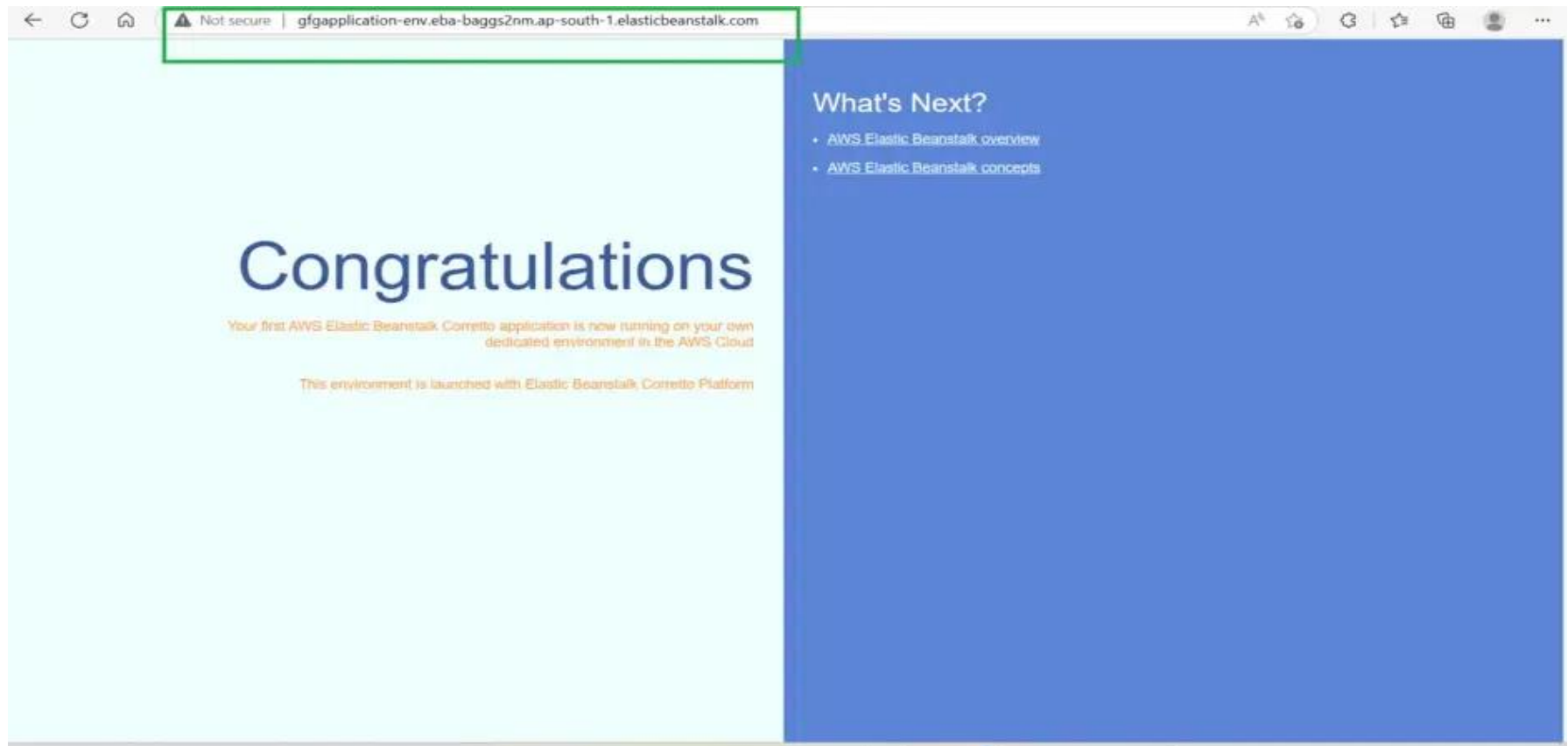
Running version
Sample Application
Upload and deploy

Platform
Corretto 17 running on 64bit Amazon Linux 2/3.4.6
Change

Recent events

Time	Type	Details
2023-04-12 12:54:51 UTC+0530	INFO	Environment health has transitioned from Pending to Ok. Initialization completed 17 seconds ago and took 3 minutes.
2023-04-12 12:54:51 UTC+0530	INFO	Added instance [i-0e4d1d652b7aad19a] to your environment.
2023-04-12 12:54:39 UTC+0530	INFO	Successfully launched environment: Gfgapplication-env
2023-04-12 12:54:39 UTC+0530	INFO	Application available at Gfgapplication-env.eba-baggs2nm.ap-south-1.elasticbeanstalk.com.
2023-04-12 12:54:23 UTC+0530	INFO	Instance deployment completed successfully.

Step 6: The very last and most important step requires us to use the URL provided by the Elastic Beanstalk to visit our Java web application.



Ways to Interact with Elastic Beanstalk

You can manage your applications in several ways:

AWS Management Console: The graphical web interface, perfect for getting started and for visual monitoring.

EB CLI (Command Line Interface): A powerful tool for developers to manage environments and deploy applications directly from their terminal. Common commands include:

- `eb init`: Initializes a project directory.

- `eb create`: Creates a new environment.

- `eb deploy`: Deploys a new version of your application.

AWS SDKs: Allow you to programmatically manage your environments from your own code.

Key Takeaways :

- **EC2 (Elastic Compute Cloud):** Provides Infrastructure as a Service (IaaS) via virtual machines; it offers the highest level of control, requiring you to manage the Operating System, security patches, and software.
- **AMI (Amazon Machine Image):** Serves as a pre-configured template (containing the OS and software) used to launch EC2 instances quickly and consistently.
- **Elastic Load Balancing (ELB):** Acts as a traffic manager that distributes incoming requests across multiple EC2 instances to ensure high availability and prevent any single server from overloading.
- **Auto Scaling:** Automatically adjusts the number of EC2 instances in your fleet based on real-time demand, ensuring you have enough power during spikes and save money during lulls.
- **AWS Lambda:** A Serverless compute service that runs code only in response to triggers (events); you never manage servers and only pay for the exact milliseconds your code executes.

- **Lambda Characteristics:** It is highly scalable, event-driven, and stateless, meaning it scales instantly to handle thousands of requests but does not store data locally between runs.
- **AWS Elastic Beanstalk:** A Platform as a Service (PaaS) that allows developers to upload their code while AWS automatically handles the deployment, load balancing, and auto-scaling of the infrastructure.
- **Compute Comparison:** Choose EC2 for total control, Elastic Beanstalk for fast web app deployment with less management, and Lambda for cost-efficient, event-driven background tasks.

Parul[®]
University

NAAC
GRADE **A++**



<https://paruluniversity.ac.in/>

